# PHYS551: Advanced Monte Carlo Methods

*Homework 2*

*Student:*
Ömer Faruk Avci

*Instructor:*
Prof. Dr. Taylan Akdoğan

Department of Electrical and Electronics Engineering

March 10, 2026

# Problem 1: The Custom Generator (Inverse Transform)

## 1. Analytical Derivation

The derivation for the Inverse Transform method is done by hand and is shown in the figure below.

1.a)

$$F(x) = \int_0^x p(x')\,dx' = \int_0^x t\,e^{-\frac{t^2}{2}}\,dt \quad \text{(substitute } x' \text{ with } t\text{)}$$

$$\frac{-t^2}{2} = u \xrightarrow{d} \underline{-t\,dt = du} \qquad t=0,\ u=0 \qquad t=x,\ u=\frac{-x^2}{2}$$

step1

$$-\int_0^{-x^2/2} e^u\,du = -e^u \Big|_0^{-x^2/2} = \Big[-e^{\frac{-x^2}{2}} \underbrace{-(-e^0)}_{1}\Big]$$

$$\boxed{F(x) = 1 - e^{\frac{-x^2}{2}}}$$

step2

$$u = 1 - e^{\frac{-x^2}{2}} \rightarrow 1-u = e^{\frac{-x^2}{2}} \rightarrow \frac{-x^2}{2} = \ln(1-u)$$

$$\downarrow$$

$$\boxed{x = \sqrt{-2\ln(1-u)}} \leftarrow -x^2 = 2\ln(1-u)$$

$$\boxed{F^{-1}(u) = \sqrt{-2\ln(1-u)}}$$

since u is uniformly distributed so 1-u is. therefore we can rewrite as:

$$\boxed{F^{-1}(u) = \sqrt{-2\ln(u)}}$$

Figure 1: Handwritten derivation for Inverse Transform of Rayleigh Distribution

## 2. Implementation

Based on the analytical derivation above, the random variable $X$ following a Rayleigh distribution can be generated by applying the transformation $x = \sqrt{-2\ln(U)}$ to a standard uniform random variable $U \sim \text{Uniform}(0, 1)$.

The following implementation defines a generator function and verifies it by comparing the empirical distribution of $10,000$ samples against the theoretical probability density function (PDF).

**Python Source Code for Problem 1**

```python
"""

# PHYS 551: Advanced Monte Carlo Methods
# Homework 2 - Source Code
# Student: Omer Faruk Avci
# Instructor: Prof. Dr. Taylan Akdogan

"""

import numpy as np
import matplotlib.pyplot as plt


"""Problem 1: The Custom Generator (Inverse Transform)"""

# Part (b): Function to generate N Rayleigh samples
def generate_rayleigh(N):
    """Generates N random samples from a Rayleigh distribution using Inverse Transform."""
    N_sample = np.random.uniform(0, 1, N)
    return np.sqrt(-2 * np.log(N_sample))

# Part (c): Function to compute the empirical CDF and showing theoretical CDF
def plot_cdf(N):

    samples = generate_rayleigh(N) # Generate N Rayleigh samples from the function defined in part (b)
    plt.hist(samples, bins=50, density=True, alpha=0.6, color='b', label='Empirical PDF (Histogram)')

    x_values = np.linspace(0, 6, 500) # Generate x values for plotting the theoretical PDF

    # p(x) = x * exp(-x^2 / 2) for x >= 0
    pdf_theoretical = x_values * np.exp(-x_values**2 / 2)

    plt.plot(x_values, pdf_theoretical, lw=2,color='red', label='Theoretical PDF $p(x)$')

    plt.title('Verification of Rayleigh Distribution Generator')
    plt.xlabel('x')
    plt.ylabel('Probability Density')
    plt.legend()
    plt.grid(axis='y', alpha=0.3)
    plt.show()

def problem_1_b():
    samples = generate_rayleigh(10000)  # Generate 10,000 samples
    print("First 10 samples from Rayleigh distribution:", samples[:10])

def problem_1_c():
    N = 10000
    plot_cdf(N)


def run_problem_1():
    print("Problem 1(b): Generating Rayleigh samples")
    problem_1_b()
    print("\nProblem 1(c): Plotting PDF")
    problem_1_c()

def main():
    run_problem_1()

main()
```

## 3. Verification and Results

To verify the accuracy of the generator, $N = 10,000$ samples were generated and plotted as a normalized histogram. The theoretical PDF, $p(x) = xe^{-x^2/2}$, is overlaid as a solid red line for comparison.

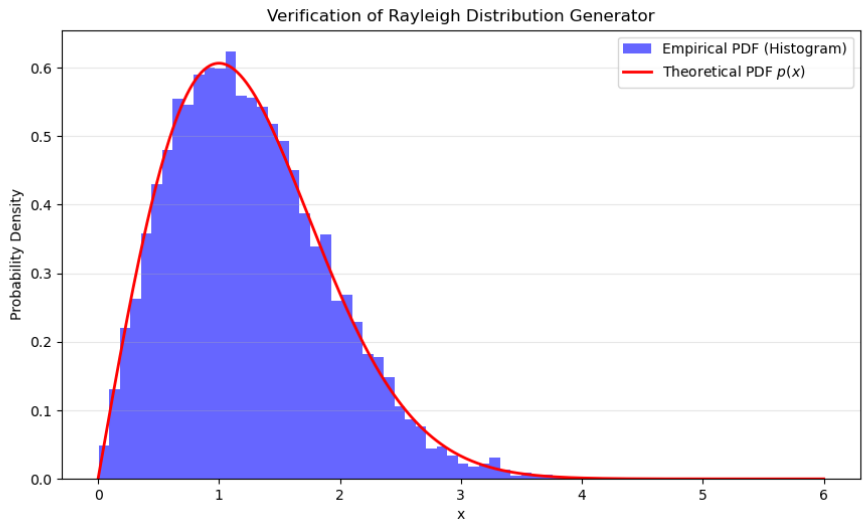


Figure 2: Normalized histogram of 10,000 Rayleigh samples vs. Theoretical PDF.

**Observations:** As shown in Figure 2, the empirical distribution of the generated samples (blue bars) closely matches the theoretical Rayleigh density function (red curve). The peak of the distribution occurs near $x = 1$, which is consistent with the mode of the Rayleigh distribution.

# Problem 2: Working Smarter (Variance Reduction)

In this problem, we estimate the integral $I = \int_0^1 e^{x^2} dx \approx 1.46265$ using three different Monte Carlo approaches with $N = 10,000$ samples.

## 1. Control Variates: Analytical Derivation

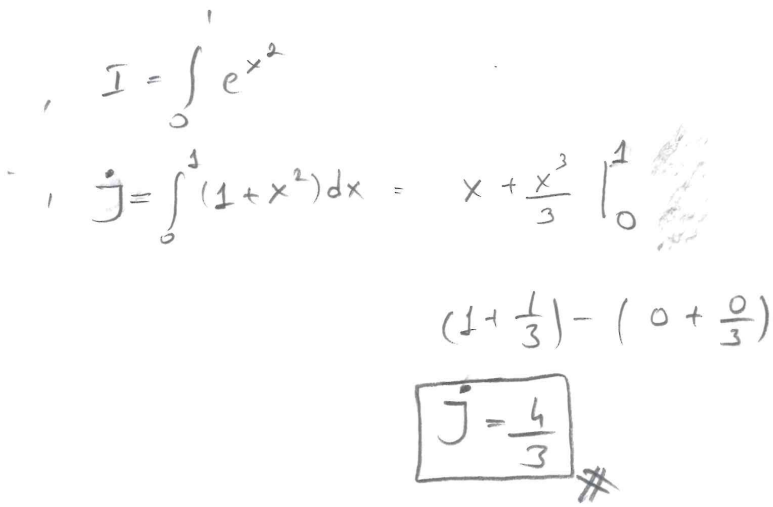


Figure 3: Handwritten derivation for Control Variates Method

## 2. Implementation

The three methods (Naive, Antithetic, and Control Variates) were implemented in Python. The Antithetic method utilizes pairs $(U, 1 - U)$ to introduce negative correlation, while the Control Variates method uses the derivation above to reduce variance.

**Python Source Code for Problem 2**

```python
"""

# PHYS 551: Advanced Monte Carlo Methods
# Homework 2 - Source Code
# Student: Omer Faruk Avci
# Instructor: Prof. Dr. Taylan Akdogan

"""

import numpy as np
import matplotlib.pyplot as plt


"""Problem 2: Working Smarter (Variance Reduction)"""

# Part (a): Naive Monte Carlo method
def naive_method(N):
    samples = np.random.uniform(0, 1, N) # N samples from uniform distribution in [0, 1]
    result = np.exp(samples**2) # Compute f(x) = exp(x^2) for each sample

```

```python
    integral_estimate = np.mean(result)
    variance = np.var(result, ddof=1)
    sem = np.sqrt(variance / N)

    return integral_estimate, variance, sem

# Part (b): Antithetic Variates method
def antithetic_method(N):
    sample_1 = np.random.uniform(0, 1, N//2) # N/2 samples from uniform distribution in [0, 1]
    sample_2 = 1 - sample_1 # Antithetic samples

    result_1 = np.exp(sample_1**2) # Compute f(x) for the first set of samples
    result_2 = np.exp(sample_2**2) # Compute f(x) for the antithetic samples

    pair_mean = (result_1 + result_2) / 2 # Average the results of the pairs

    intgral_estimate = np.mean(pair_mean)
    variance = np.var(pair_mean, ddof=1)
    sem = np.sqrt(variance / (N//2))

    return intgral_estimate, variance, sem

# Part (c): Control Variates method
def control_variates_method(N):

    samples  = np.random.uniform(0, 1, N)

    X = np.exp(samples ** 2) # f(X) = e^(X^2)
    Y = samples ** 2 + 1 # g(X) = X^2 + 1, J = E[g(X)] = 4/3

    J = 4 / 3

    c_estimated =  np.cov(X, Y)[0, 1] / np.var(Y, ddof=1) # Estimate the optimal control variate coefficient c

    L = X - c_estimated * (Y - J) # Adjusted estimator using control variates

    mean = np.mean(L)
    variance = np.var(L, ddof=1)
    sem = np.sqrt(variance / N)

    return mean, variance, sem

def run_problem_2():
    N = 10000

    # Run all three methods and collect their estimates, variances, and SEMs
    naive_est, naive_var, naive_sem = naive_method(N)
    anti_est, anti_var, anti_sem = antithetic_method(N)
    control_est, control_var, control_sem = control_variates_method(N)

    # Calculate Variance Reduction Factors (VRF) for both methods compared to the naive method
    vrf_anti = naive_var / anti_var
    vrf_control = naive_var / control_var

    # Print the results in a formatted table
    print("\nProblem 2 Results (N = 10,000)")
    print("-" * 65)
    print(f"{'Method':<22} {'Estimate':<12} {'Variance':<14} {'SEM':<14}")
    print("-" * 65)

    print(f"{'Naive Monte Carlo':<22} {naive_est:<12.6f} {naive_var:<14.6e} {naive_sem:<14.6e}")
    print(f"{'Antithetic Variates':<22} {anti_est:<12.6f} {anti_var:<14.6e} {anti_sem:<14.6e} ")
    print(f"{'Control Variates':<22} {control_est:<12.6f} {control_var:<14.6e} {control_sem:<14.6e}")

    print("-" * 65)
    print(f"VRF (Antithetic vs Naive): {vrf_anti:.3f}")
    print(f"VRF (Control vs Naive):    {vrf_control:.3f}")
    print("-" * 65)

def main():
    run_problem_2()

main()
```

## 3. Verification and Results

**Problem 2 Results**

```
Problem 2 Results (N = 10,000)
----------------------------------------------------------------
Method Estimate Variance SEM
----------------------------------------------------------------
Naive Monte Carlo 1.460129 2.256655e-01 4.750426e-03
Antithetic Variates 1.464030 2.779314e-02 2.357674e-03
Control Variates 1.461401 3.738740e-03 6.114523e-04
----------------------------------------------------------------
VRF (Antithetic vs Naive):  8.119
VRF (Control vs Naive):  60.359
----------------------------------------------------------------
```

The results of the simulation are tabulated below. For all methods, the sample size was kept constant at $N = 10,000$ to provide a fair comparison of efficiency.

| **Method** | **Estimate** $\hat{I}$ | **Variance** ($\sigma^2$) | **SEM** | **VRF** |
|---|---|---|---|---|
| Naive Monte Carlo | 1.4601 | $225.67 \times 10^{-3}$ | $4.75 \times 10^{-3}$ | 1.00 |
| Antithetic Variates | 1.4640 | $27.79 \times 10^{-3}$ | $2.36 \times 10^{-3}$ | 8.12 |
| Control Variates | 1.4614 | $3.74 \times 10^{-3}$ | $0.61 \times 10^{-3}$ | 60.36 |

Table 1: Performance comparison of variance reduction techniques for $N = 10,000$.

**Observations:** The data clearly demonstrates that **Control Variates** is the most effective method for this integral. While the Antithetic method provided a significant gain by reducing variance by a factor of 8.12, the Control Variates method achieved a Variance Reduction Factor (VRF) of 60.36. This implies that the Control Variates estimator is roughly 60 times more computationally efficient than the Naive approach.

# Problem 3: The Curse of Dimensionality

## 1. Simulation Implementation

The estimation of a $d$-dimensional hypersphere volume is performed using rejection sampling (the "dart-throwing" method). We generate $N$ points in a $d$-dimensional hypercube of side length 2 (range $[-1, 1]$) and count how many fall within the unit distance from the origin.

**Python Source Code for Problem 3**

```python
"""

# PHYS 551: Advanced Monte Carlo Methods
# Homework 2 - Source Code
# Student: Omer Faruk Avci
# Instructor: Prof. Dr. Taylan Akdogan

"""
import numpy as np

"""Problem 3: The Curse of Dimensionality"""

def estimate_volume(d, n_samples=1_000_000):
    # Generate N points in d-dimensions within [-1, 1]
    points = np.random.uniform(-1, 1, (n_samples, d))

    # Calculate squared Euclidean distance for each point
    distances_squared = np.sum(points**2, axis=1)

    # Count points satisfying the hypersphere condition: sum(x_i^2) <= 1
    inside_count = np.sum(distances_squared <= 1)

    # Volume of the bounding hypercube is 2^d
    v_cube = 2**d

    # Scale the hit-ratio by the total volume of the cube
    estimated_volume = (inside_count / n_samples) * v_cube

    return estimated_volume

def run_problem_3():
    dimensions = [2, 5, 10]
    results = {d: estimate_volume(d) for d in dimensions}

    print(f"Problem 3 Results (N = 1,000,000 samples):")
    print("-" * 45)
    print(f"{'Dimension (d)':<15} {'Estimated Volume':<20}")
    print("-" * 45)
    for d, vol in results.items():
        print(f"{d:<15} {vol:<20.5f}")

def main():
    run_problem_3()

if __name__ == "__main__":
    main()
```

## 2. Results and Discussion

The simulation was conducted for $d \in \{2, 5, 10\}$ using $N = 10^6$ samples. The results are summarized in Table 2.

Table 2: Hypersphere Volume Estimates ($N = 1,000,000$)

| Dimension ($d$) | Estimated Volume |
|---|---|
| 2 | 3.14150 |
| 5 | 5.26384 |
| 10 | 2.54874 |

### Analysis of Findings

- **Accuracy:** The 10D estimate ($\approx 2.549$) is highly consistent with the analytical value $V_{10} = \frac{\pi^5}{120} \approx 2.550$.
- **Curse of Dimensionality:** At $d = 10$, the ratio of the hypersphere volume to the bounding hypercube ($2^{10} = 1024$) is a mere 0.249%. This illustrates that in high-dimensional spaces, volume concentrates in the "corners" of the hypercube, leaving the inscribed sphere nearly negligible.
- **Computational Efficiency:** From a Monte Carlo perspective, over 99.7% of samples are rejected at $d = 10$. Because the "hit rate" decays exponentially with $d$, the sample size $N$ must grow exponentially to maintain precision, rendering simple rejection sampling impractical for high-dimensional integration.

# Output

**Sample Terminal Output from Running the Code**

```
Problem 1(b):  Generating Rayleigh samples
First 10 samples from Rayleigh distribution:  [1.10402963 1.16078819 1.66809063 1.72289481 1.95939379 2.17591001
1.10808643 0.57249355 0.46745059 1.33945967]
Problem 1(c):  Plotting PDF
Problem 2 Results (N = 10,000)
----------------------------------------------------------------
Method Estimate Variance SEM
----------------------------------------------------------------
Naive Monte Carlo 1.451307 2.182678e-01 4.671914e-03
Antithetic Variates 1.462003 2.772500e-02 2.354782e-03
Control Variates 1.462234 3.797647e-03 6.162505e-04
----------------------------------------------------------------
VRF (Antithetic vs Naive):  7.873
VRF (Control vs Naive):  57.474
----------------------------------------------------------------
Problem 3 Results (N = 1,000,000 samples):
-------------------------------------------
Dimension (d) Estimated Volume
-------------------------------------------
2 3.14163
5 5.26656
10 2.47398
```